LABORATORY 5 · WEEK 5 · 26–30 OCTOBER 2026

Assignment A3: Files, Logging and a Command Menu

Toy-language interpreter, continued · string type · file statements · relational expressions · log file · Command pattern

#	This week
1	**Present assignment A2** orally to the lab instructor (tagged `A2`, see the checklist in the Laboratory 3 document).
2	**Start assignment A3** in the **same** Gradle project and Git repository as A2. Work in class and continue at home.
3	Check `LabAssignmentsSchedule.pdf` regularly for the assignment schedule.

> **Deadline for A3: week 7 (9–13 November 2026)**
>
> A3 is **presented orally to the lab instructor during the laboratory**: you pull the version tagged `A3`, run `./gradlew build` (compilation + all tests, the A2 tests included), run the examples from the menu – among them the file example – open the log file and explain your design and code.

Overview

A3 extends the interpreter from A2. All the A2 code stays; you add four features:

#	Feature	What you add or change
1	**Files in the toy language**	`StringType`, `StringValue`, the **FileTable** in `PrgState`, the statements `openRFile`, `readFile`, `closeRFile`
2	**Relational expressions**	`RelOp`, `RelationalExp`: `<`, `<=`, `==`, `!=`, `>`, `>=`
3	**Log file**	`IRepository.logPrgStateExec`, `Repository`, `Controller.allStep`
4	**Command-based text menu**	`Command`, `ExitCommand`, `RunExampleCommand`, `TextMenu`, `Main`

> **Already done in A2**
>
> `IType` already declares `defaultValue()` and `VarDeclStmt` already uses it (rule S3). Types and values are **records**, so `equals` and `hashCode` already compare by content. Check that this holds in your code.
>
> If you wrote values as ordinary classes, override **both** `equals` and `hashCode` in `IntValue`, `BoolValue` and `StringValue` now: a `StringValue` is used as a **key** of a hash map (the FileTable), and without `hashCode` the lookup fails.

Suggested order of work

Work in small steps. Each step ends with green tests (`./gradlew test`) and a commit.

Step	Done when	Tests
1. String type	`StringType`, `StringValue`; `string` variables get the default value `""`	default value; `equals` and `hashCode` of string values
2. Relational expressions	`RelOp`, `RelationalExp`	each operator, true and false; non-integer operand

Step	Done when	Tests
3. FileTable	new ADT operations; `PrgState` has a FileTable; all `new PrgState(…)` calls updated	all A2 tests still pass
4. File statements	`OpenRFileStmt`, `ReadFileStmt`, `CloseRFileStmt` (rules S8–S10)	each rule and each of its errors
5. Log file	`logPrgStateExec`; `allStep` logs every state	the log contains the four sections
6. Command menu	`TextMenu`, commands, new `Main`	– (run with `./gradlew run`)
7. Examples	Ex4 (files) and Ex5 (relational) in the menu	Ex4 prints 15, 50; with an empty file 0, 0; Ex5 prints 5

1. Language extensions

1.1 Syntax

The grammar from A2 gets a new type, a new kind of value, three statements and the relational expressions:

`grammar – new parts`

```
Stmt  ::= …                                  (the six statements of A2)
        | openRFile(Exp)                     (OpenRFileStmt)
        | readFile(Exp, Id)                  (ReadFileStmt)
        | closeRFile(Exp)                    (CloseRFileStmt)

Type  ::= int | bool | string                (StringType)

Value ::= Number | true | false
        | "text"                             (StringValue)

Exp   ::= …                                  (the expressions of A2)
        | Exp1 <  Exp2 | Exp1 <= Exp2        (RelationalExp)
        | Exp1 == Exp2 | Exp1 != Exp2
        | Exp1 >  Exp2 | Exp1 >= Exp2
```

1.2 Program state

The program state gets a fourth structure, the **FileTable**:

Structure	Content	Notation
ExeStack	the statements that still have to be executed	`{s1 | s2 | s3}`
SymTable	the declared variables and their values	`{varf -> "test.in", varc -> 15}`
Out	the printed values	`{15, 50}`
FileTable	*new:* the files opened by the program – the file name (a `StringValue`) mapped to the Java `BufferedReader` that reads it	`{"test.in" -> reader}`

We assume that a file read by a toy program is a **text file with one positive integer per line**, for example:

`test.in`

```
15
50
```

1.3 Execution rules

The rules S1–S7 and E1–E4 of A2 stay unchanged; rule S3 gives a `string` variable the default value `""`. As before, the controller pops the statement before applying its rule.

Rule	Top of ExeStack	Effect	Errors
S8	`openRFile(Exp)`	`f = Eval(Exp)`; open the file named `f` with a `BufferedReader`; add `f -> reader` to FileTable	`f` is not a string; `f` is already in FileTable; the file does not exist or cannot be opened
S9	`readFile(Exp, Id)`	`f = Eval(Exp)`; read a line with the reader of `f`; `n = 0` if the line is `null` (end of file), otherwise `n = Integer.parseInt(line)`; update `Id -> n` in SymTable	`Id` not declared or not of type `int`; `f` is not a string; `f` is not in FileTable; the line is not an integer; I/O error
S10	`closeRFile(Exp)`	`f = Eval(Exp)`; close the reader of `f`; remove `f` from FileTable	`f` is not a string; `f` is not in FileTable; I/O error

Rule	Expression	Value	Errors
E5	`Exp1 op Exp2`, `op` ∈ {`<`, `<=`, `==`, `!=`, `>`, `>=`}	evaluate `Exp1`, then `Exp2`; both must be `int`; the result is a `bool`	operand 1 / operand 2 is not an integer

1.4 New example programs

	Program	Output
Ex4	`string varf; varf = "test.in"; openRFile(varf); int varc; readFile(varf, varc); print(varc); readFile(varf, varc); print(varc); closeRFile(varf)`	`15`, `50` – with an empty `test.in`: `0`, `0`
Ex5	`int a; int b; a = 5; b = 7; (if a < b then print(a) else print(b))`	`5`

2. Implementation requirements

The fragments below continue the design of A2: records for types, values, expressions and statements, enums for operators, interfaces starting with **I**. Complete the parts marked *TODO*.

2.1 The string type and string values

A new type is simply a new implementation of `IType` – a record, like `IntType` and `BoolType`. Nothing in the existing code has to change: this is the *design for extension* from A2.

`model/type/StringType.java, model/value/StringValue.java`

```
public record StringType() implements IType {
    @Override public IValue defaultValue() { return new StringValue(""); }
    @Override public String toString() { return "string"; }
}

public record StringValue(String val) implements IValue {
    @Override public IType getType() { return new StringType(); }
    @Override public String toString() { return "\"" + val + "\""; }
}
```

2.2 Relational expressions

One class for all six operators, with the operator as an enum – exactly like `ArithExp` and `ArithOp`. The result is a `BoolValue`, so relational expressions can be used as `if` conditions.

model/exp

```
public enum RelOp {
    LT("<"), LE("<="), EQ("=="), NE("!="), GT(">"), GE(">=");
    private final String symbol;
    RelOp(String symbol) { this.symbol = symbol; }
    @Override public String toString() { return symbol; }
}

public record RelationalExp(RelOp op, IExp left, IExp right) implements IExp {
    @Override
    public IValue eval(IDictionary<String, IValue> symTable) throws MyException {
        if (!(left.eval(symTable) instanceof IntValue(int n1)))
            throw new ExpressionException("first operand is not an integer");
        if (!(right.eval(symTable) instanceof IntValue(int n2)))
            throw new ExpressionException("second operand is not an integer");
        return new BoolValue(switch (op) {
            case LT -> n1 < n2;
            case LE -> n1 <= n2;
            // TODO: EQ, NE, GT, GE
        });
    }
    @Override
    public String toString() { return "(" + left + " " + op + " " + right + ")"; }
}
```

2.3 The FileTable

The FileTable is a dictionary, so it **reuses the generic ADT from A2**: its type is `IDictionary<StringValue, BufferedReader>`. No new ADT class is needed – only two new operations, used by `closeRFile` and by the log file. The stack and the list also get a way to list their elements.

model/adt

```
public interface IDictionary<K, V> {
    boolean isDefined(K key);
    V lookup(K key) throws ADTException;
    void put(K key, V value);
    void update(K key, V value) throws ADTException;
    void remove(K key) throws ADTException;     // new: missing key -> ADTException
    Map<K, V> toMap();                          // new: a read-only copy (Map.copyOf)
}

public interface IStack<T> { … List<T> toList(); }  // new: top of the stack first
public interface IList<T>  { … List<T> toList(); }  // new: in insertion order
```

`PrgState` gets the FileTable as a new field. Add also a **static factory** that creates a state with empty structures:

model/PrgState.java

```
public class PrgState {
    private IStack<IStmt> exeStack;
    private IDictionary<String, IValue> symTable;
    private IList<IValue> out;
    private IDictionary<StringValue, BufferedReader> fileTable;        // new in A3
    private final IStmt originalProgram;

    public PrgState(IStack<IStmt> exeStack, IDictionary<String, IValue> symTable,
                    IList<IValue> out,
                    IDictionary<StringValue, BufferedReader> fileTable,
                    IStmt program) { … }

    public static PrgState of(IStmt program) {
        return new PrgState(new MyStack<>(), new MyDictionary<>(), new MyList<>(),
                            new MyDictionary<>(), program);
    }
    // getters and setters for the four structures, toString, toLogString (2.6)
}
```

> **Why a factory method?**
>
> Adding the FileTable changes the constructor, so every `new PrgState(…)` in `Main`, in the examples and in the tests must change. Replace them with `PrgState.of(program)`: when a later assignment adds another structure, only the factory changes.

2.4 File statements

`openRFile` is given in full; write `readFile` and `closeRFile` in the same style, following rules S9 and S10.

model/stmt

```
public record OpenRFileStmt(IExp exp) implements IStmt {                    // S8
    @Override
    public PrgState execute(PrgState state) throws MyException {
        if (!(exp.eval(state.getSymTable()) instanceof StringValue fileName))
            throw new StatementException("openRFile: " + exp + " is not a string");
        var fileTable = state.getFileTable();
        if (fileTable.isDefined(fileName))
            throw new StatementException("openRFile: " + fileName + " is already open");
        try {
            // no try-with-resources: the file must stay open until closeRFile
            var reader = new BufferedReader(new FileReader(fileName.val()));
            fileTable.put(fileName, reader);
        } catch (IOException e) {
            throw new StatementException("openRFile: cannot open " + fileName
                    + ": " + e.getMessage());
        }
        return state;
    }
    @Override public String toString() { return "openRFile(" + exp + ")"; }
}

// TODO: ReadFileStmt(IExp exp, String varName)    – S9
//         check varName (declared, type int) and the file (string, in FileTable);
//         readLine(): null -> 0, otherwise Integer.parseInt(line.strip());
//         turn IOException and NumberFormatException into StatementException
// TODO: CloseRFileStmt(IExp exp)                  – S10
//         close() the reader, then fileTable.remove(fileName)
```

> **Java exceptions stop at the model boundary**
>
> `IOException` and `NumberFormatException` are Java exceptions; the interpreter reports its own. Catch them **inside the statement** and throw a `StatementException` with a clear message – the controller and the view only ever see `MyException`.

> **Where are relative file names resolved?**
>
> A file name such as `"test.in"` is relative to the **working directory** of the program. With `./gradlew run` that is the `app/` folder. Simplest: run from the project root, so `test.in` and the log files stay next to `settings.gradle.kts` – add one line to the `run` task, and add `log*.txt` to `.gitignore`:

app/build.gradle.kts

```
tasks.named<JavaExec>("run") {
    standardInput = System.`in`
    workingDir = rootDir          // new: test.in and log files in the project root
}
```

2.5 Statements and expressions in infix form

Statements and expressions are **binary trees** of objects. Their `toString` must traverse the tree **left – root – right** and produce the program in its usual (infix) form, for example `(a = (2 + (3 * 5)); print(a))`. The records written as in A2 already do this; check that every new class does too – the log file and the menu use these strings.

2.6 The log file

After every step the repository appends the current program state to a text file. Add the method to the interface and implement it in `Repository`; the path of the log file is a constructor parameter (optionally, `Main` can read it from the keyboard with a `Scanner`).

repository

```
public interface IRepository {
    PrgState getCrtPrg();
    void logPrgStateExec() throws MyException;                  // new in A3
}

public class Repository implements IRepository {
    private final List<PrgState> states = new ArrayList<>();
    private final String logFilePath;                           // new in A3

    public Repository(PrgState initial, String logFilePath) {
        states.add(initial);
        this.logFilePath = logFilePath;
    }

    @Override public PrgState getCrtPrg() { return states.getFirst(); }

    @Override
    public void logPrgStateExec() throws MyException {
        try (var logFile = new PrintWriter(
                new BufferedWriter(new FileWriter(logFilePath, true)))) { // append
            logFile.println(getCrtPrg().toLogString());
        } catch (IOException e) {
            throw new RepositoryException("cannot write the log file " + logFilePath
                    + ": " + e.getMessage());
        }
    }
}
```

Here **try-with-resources** is right: the log file is opened, written and closed at every call, so nothing is lost if the program stops with an error. Each state is written in the following format – the section names must appear exactly like this, and each element is on its own line, the **top of the stack first**:

`model/PrgState.java`

```
public String toLogString() {
    var sb = new StringBuilder();
    sb.append("ExeStack:\n");
    exeStack.toList().forEach(s -> sb.append(s).append('\n'));
    sb.append("SymTable:\n");
    symTable.toMap().forEach((k, v) ->
            sb.append(k).append(" --> ").append(v).append('\n'));
    sb.append("Out:\n");
    out.toList().forEach(v -> sb.append(v).append('\n'));
    sb.append("FileTable:\n");
    fileTable.toMap().keySet().forEach(f -> sb.append(f.val()).append('\n'));
    return sb.toString();
}
```

The controller writes the initial state and the state after each step:

`controller/Controller.java`

```
public void allStep() throws MyException {
    PrgState prg = repo.getCrtPrg();
    repo.logPrgStateExec();
    display(prg);
    while (!prg.getExeStack().isEmpty()) {
        oneStep(prg);
        repo.logPrgStateExec();
        display(prg);
    }
}
```

An excerpt of `log4.txt` for Ex4 – the initial state, the state after `openRFile(varf)` and the state just before the first `readFile`:

`log4.txt (excerpt)`

```
ExeStack:
(string varf; (varf = "test.in"; (openRFile(varf); (int varc; …))))
SymTable:
Out:
FileTable:

ExeStack:
(int varc; (readFile(varf, varc); (print(varc); (readFile(varf, varc); …))))
SymTable:
varf --> "test.in"
Out:
FileTable:
test.in

ExeStack:
readFile(varf, varc)
(print(varc); (readFile(varf, varc); (print(varc); closeRFile(varf))))
SymTable:
varf --> "test.in"
varc --> 0
Out:
FileTable:
test.in
```

Note: the file is opened in append mode, so running the program again adds to the old log. Delete the log files before a demonstration.

2.7 The view: a menu built from commands

The menu of A2 is replaced by a **text menu built from command objects** (the *Command* design pattern): each menu option is an object with a key, a description and an `execute` method. Adding an option means adding a command – the menu itself does not change.

view/Command.java, ExitCommand.java, RunExampleCommand.java

```
public abstract class Command {
    private final String key;
    private final String description;

    protected Command(String key, String description) {
        this.key = key;
        this.description = description;
    }
    public abstract void execute();
    public String getKey() { return key; }
    public String getDescription() { return description; }
}

public class ExitCommand extends Command {
    public ExitCommand(String key, String description) { super(key, description); }
    @Override public void execute() { System.exit(0); }
}

public class RunExampleCommand extends Command {
    private final Controller controller;
    private boolean executed = false;

    public RunExampleCommand(String key, String description, Controller controller) {
        super(key, description);
        this.controller = controller;
    }

    @Override
    public void execute() {
        if (executed) {                          // an example runs only once
            System.out.println("This example has already been executed.");
            return;
        }
        executed = true;
        try {
            controller.allStep();
        } catch (MyException e) {                // errors the controller cannot solve
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

view/TextMenu.java

```
public class TextMenu {
    private final Map<String, Command> commands = new LinkedHashMap<>(); // in order

    public void addCommand(Command c) { commands.put(c.getKey(), c); }

    private void printMenu() {
        for (Command c : commands.values())
            System.out.printf("%4s : %s%n", c.getKey(), c.getDescription());
    }

    public void show() {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            printMenu();
            System.out.print("Input the option: ");
            Command c = commands.get(scanner.nextLine().strip());
            if (c == null) {
                System.out.println("Invalid option");
                continue;
            }
            c.execute();
        }
    }
```

```
}
```

`Main` creates one program state, one repository (with its own log file) and one controller per example, and one command for each:

`Main.java`

```
public class Main {
    public static void main(String[] args) {
        List<IStmt> programs = List.of(Examples.EX1, Examples.EX2, Examples.EX3,
                                       Examples.EX4, Examples.EX5);
        TextMenu menu = new TextMenu();
        menu.addCommand(new ExitCommand("0", "exit"));
        for (int i = 0; i < programs.size(); i++) {
            IStmt program = programs.get(i);
            var repo = new Repository(PrgState.of(program), "log" + (i + 1) + ".txt");
            var controller = new Controller(repo, true);
            menu.addCommand(new RunExampleCommand(String.valueOf(i + 1),
                                                  program.toString(), controller));
        }
        menu.show();
    }
}
```

`./gradlew run`

```
   0 : exit
   1 : (int v; (v = 2; print(v)))
   2 : (int a; (int b; (a = (2 + (3 * 5)); (b = (a + 1); print(b)))))
   3 : (bool a; (int v; (a = true; ((if (a) then (v = 2) else (v = 3)); print(v)))))
   4 : (string varf; (varf = "test.in"; (openRFile(varf); (int varc; …))))
   5 : (int a; (int b; (a = 5; (b = 7; (if ((a < b)) then (print(a)) else …)))))
Input the option:
```

2.8 Example programs and exceptions

Add Ex4 and Ex5 to `view.Examples`. Write the file example as a **method with the file name as parameter**, so that the tests can run it on temporary files:

`view/Examples.java`

```
public static IStmt fileExample(String fileName) {
    return new CompStmt(new VarDeclStmt("varf", new StringType()),
        new CompStmt(new AssignStmt("varf", new ValueExp(new StringValue(fileName))),
        new CompStmt(new OpenRFileStmt(new VarExp("varf")),
        new CompStmt(new VarDeclStmt("varc", new IntType()),
        new CompStmt(new ReadFileStmt(new VarExp("varf"), "varc"),
        new CompStmt(new PrintStmt(new VarExp("varc")),
        new CompStmt(new ReadFileStmt(new VarExp("varf"), "varc"),
        new CompStmt(new PrintStmt(new VarExp("varc")),
                     new CloseRFileStmt(new VarExp("varf")))))))));
}

public static final IStmt EX4 = fileExample("test.in");

// TODO: EX5 – int a; int b; a = 5; b = 7;
//             (if a < b then print(a) else print(b))
```

Extend the exception hierarchy with a `RepositoryException` (subclass of `MyException`) for errors of the log file. The new errors of rules S8–S10 are `StatementException`s, those of rule E5 `ExpressionException`s, and `remove` of a missing key is an `ADTException`.

3. Testing with JUnit

All the A2 tests must still pass – after you replace `new PrgState(…)` with `PrgState.of(…)` and give every `Repository` a log file. Add tests for every new feature.

3.1 Minimum tests for A3

Area	Test at least
Strings	a `string` variable gets the default value `""`; two equal `StringValue`s are `equals` and have the same `hashCode`
Relational expressions	each of the six operators, with a true and a false result; non-integer operand throws
openRFile	adds the file to the FileTable; non-string expression, file already open and missing file throw
readFile	reads the numbers in order and gives `0` at the end of the file; variable not declared or not `int`, file not open and non-integer line throw
closeRFile	removes the file from the FileTable; closing a file that is not open throws
Log file	after `allStep` the log contains `ExeStack:`, `SymTable:`, `Out:`, `FileTable:` in this order
Controller	Ex4 prints `15`, `50` and ends with an empty FileTable; with an empty file it prints `0`, `0`; Ex5 prints `5`

3.2 Tests with files: @TempDir

Tests must not depend on files in your project folder. JUnit's `@TempDir` gives each test a **fresh temporary folder** that is deleted afterwards; the test writes its input files there with `Files.writeString` (NIO.2, Lecture 5).

`src/test/java/interpreter/model/stmt/FileStmtTest.java`

```
package interpreter.model.stmt;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import java.nio.file.Files;
import java.nio.file.Path;
// + imports of the interpreter classes

class FileStmtTest {
    @TempDir
    Path dir;                                   // a new empty folder for every test

    private PrgState state;
    private ValueExp file;                      // the expression "<dir>/numbers.in"

    @BeforeEach
    void setUp() throws Exception {
        Path f = dir.resolve("numbers.in");
        Files.writeString(f, "7\n");
        file = new ValueExp(new StringValue(f.toString()));
        state = PrgState.of(new NopStmt());
        new VarDeclStmt("x", new IntType()).execute(state);
    }

    @Test
    void readGivesTheNextNumberThenZeroAtTheEnd() throws MyException {
        new OpenRFileStmt(file).execute(state);
        new ReadFileStmt(file, "x").execute(state);
        assertEquals(new IntValue(7), state.getSymTable().lookup("x"));
        new ReadFileStmt(file, "x").execute(state);
```

```
        assertEquals(new IntValue(0), state.getSymTable().lookup("x"));
    }

    @Test
    void openingTheSameFileTwiceThrows() throws MyException {
        new OpenRFileStmt(file).execute(state);
        assertThrows(StatementException.class,
                     () -> new OpenRFileStmt(file).execute(state));
    }
}
```

src/test/java/interpreter/controller/ControllerTest.java

```
class ControllerTest {
    @TempDir
    Path dir;

    private PrgState run(IStmt program) throws MyException {
        PrgState state = PrgState.of(program);
        var repo = new Repository(state, dir.resolve("log.txt").toString());
        new Controller(repo, false).allStep();                    // display off
        return state;
    }

    @Test
    void fileExampleReadsBothNumbersAndClosesTheFile() throws Exception {
        Path in = dir.resolve("test.in");
        Files.writeString(in, "15\n50\n");
        PrgState s = run(Examples.fileExample(in.toString()));
        assertEquals(new IntValue(15), s.getOut().get(0));
        assertEquals(new IntValue(50), s.getOut().get(1));
        assertTrue(s.getFileTable().toMap().isEmpty());
    }

    @Test
    void logFileContainsTheFourSectionsInOrder() throws Exception {
        run(Examples.EX1);
        String log = Files.readString(dir.resolve("log.txt"));
        int a = log.indexOf("ExeStack:"), b = log.indexOf("SymTable:"),
            c = log.indexOf("Out:"),      d = log.indexOf("FileTable:");
        assertTrue(0 <= a && a < b && b < c && c < d);
    }
}
```

4. Submission checklist (A3)

- ☐ A3 is implemented in the A2 project and repository; `./gradlew build` succeeds on a fresh clone.
- ☐ `StringType`, `StringValue`, `RelationalExp`, the FileTable and the statements `openRFile`, `readFile`, `closeRFile` follow sections 1 and 2.
- ☐ Every error of rules S8–S10 and E5 produces an interpreter exception with a clear message; no Java exception reaches the menu.
- ☐ The log file has the format of section 2.6 and is written after every step.
- ☐ The menu is built from commands; examples Ex1–Ex5 run from it with `./gradlew run`, each only once.
- ☐ The A2 tests and the new tests of section 3.1 pass; file tests use `@TempDir`.
- ☐ The Git history shows incremental work with meaningful messages; log files and `build/` are not committed.
- ☐ The submitted version is tagged `A3` (`git tag A3`, `git push --tags`) before the deadline.

☐ **The project is presented orally to the lab instructor at the laboratory:** you run the build, the tests and the examples, show a log file and explain the design and any part of your code. Code you cannot explain is not accepted.

Deadline: week 7 (9–13 November 2026).